1. input() Function

In [ ]:
```
name = input("Enter your name: ")
print(name)

age = int(input("Enter your age: "))
print(age)
```

In [ ]:
```
# syntax:

# variable = input("Enter something: ")
```

In [ ]:
```
# Simple input function
a=input()
print(a)
```

In [ ]:
```
# Simple Input
name = input("Enter your name: ")
print("Hello,", name)
```

In [ ]:
```
a=input('Enter the value of a:')
b=input('Enter the value of b:')
total=a+b
print(f'The sum of {a} and {b} is {total}')
```

In [ ]:
```
# Adding the integer values and displaying the output as float value
a=int(input('Enter the value of a:'))
b=int(input('Enter the value of b:'))
total=a+b
print(f'The sum of {a} and {b} is {float(total)}')
```

In [ ]:
```
age = input("Enter your age: ")
print(type(age))  # <class 'str'>

# Convert to integer
age = int(age)
print("You will be", age + 1, "next year.")
```

2. String Indexing and Slicing

In [ ]:
```
text = "Python"
print(text[0])
print(text[-1])
print(text[0:3])
print(text[::2])
```

In [ ]:
```
str1 = 'Hello'
str2 = "World"
str3 = '''This is a multi-line string'''
```

In [ ]:
```
str1
```

In [ ]:
```
str2
```

In [ ]:
```
str3
```

In [ ]:
```
doc = "It's a fine day."
doc
```

In [ ]:
```
print("Type of the document:",type(doc))
```

In [ ]:
```
address = '#2 Main Street, 56/3' # Define string using single quotes
print(address)
```

In [ ]:
```
str1 = "HELLO PYTHON"
print(str1)
```

In [ ]:
```
str1[0] # First character in string "str1"
```

In [ ]:
```
str1[len(str1)-1] # Last character in string using len function
```

In [ ]:
```
str1[3]
```

In [ ]:
```
str1[7]
```

In [ ]:
```
str1[-1] # Last character
```

In [ ]:
```
str1[-7] # reverse indexing possible
```

In [ ]:
```
str1[-8]
```

In [ ]:
```
str1[0:5] ## start:end   # start from 0 to 5 index position exclusive(5-1
```

In [ ]:
```
str1[6:12] # String slicing - Retreive all characters between 6 - 12 inde
```

In [ ]:
```
str1[:] # whole string
```

In [ ]:
```
str1[3:] # default end ~ end of string
```

In [ ]:
```
str1[:6] # default start ~ beginning of string
```

In [ ]:
```
str1[-10:-5]
```

In [ ]:
```
str1[-4:] # Retreive last four characters of the string
```

In [ ]:
```
str1[-6:] # Retreive last six characters of the string
```

In [ ]:
```
# Slice with step: every second character ( skip one value )
print(str1[::2])    # Output: "HLOPTO"
```

In [ ]:
```
#slice with step : every third character (skip two value )
print(str1[::3])   # Output "HLPH"
```

In [ ]:
```
s = "hello"
s[0] = "H"   # ❌ TypeError: 'str' object does not support item assignment
```

In [ ]:
```
# To modify a string, you need to create a new one:
s = "hello"
s = "H" + s[1:]
print(s)  # Output: "Hello"
```

In [ ]:
```
course='Data Science'
course[:4]
```

In [ ]:
```
course[:4] = "Life"
```

In [ ]:
```
# Alternately try with the replace function
course.replace("Data","Life")
```

In [ ]:
```
# while it delivers the output
print(course) # the immutable string remains unchanged
```

In [ ]:
```
del course # delete the object
# called right after - error confirms success
```

In [ ]:
```
course
```

3. String Functions

In [ ]:
```
text = " hello python "
print(text.upper())
print(text.lower())
print(text.strip())
print(text.replace("python", "world"))
```

In [ ]:
```
s1 = "Well"
s2 = "being"
s1 + s2
```

In [ ]:
```
# storing the result
s3 = s1 + "-" + s2
s3
```

In [ ]:
```
# String repetition
str= 'hi'
result=str*5
result
```

In [ ]:
```
str= 'hi '
result=str*5
result
```

In [ ]:
```
str='hello'
result=(str+' ')*5
result
```

In [ ]:
```
# To count the number of characters present inside the string
str1='Hello World'
len(str1)
```

In [ ]:
```
### String Membership
mystr1 = "Hello Everyone"
print ('Hello' in mystr1) # Check whether substring "Hello" is present in
print ('Everyone' in mystr1) # Check whether substring "Everyone" is pres
print ('Hi' in mystr1) # Check whether substring "Hi" is present in strin
```

In [ ]:
```
str5 = "Natural language processing with Python and R and Java"
L = str5.partition("and")
L
```

In [ ]:
```
str5.count("and")
```

In [ ]:
```
# example :
# lower
s='hello PYTHON'
s.lower()
```

In [ ]:
```
#uppercase:
s.upper()
```

In [ ]:
```
# capitalize
s.capitalize()
```

In [ ]:
```
# title
s.title()
```

In [ ]:
```
s.swapcase()
```

In [ ]:
```
# find method
"banana".find("xx")
```

In [ ]:
```
# rfind method
"banana".rfind("na")
```

In [ ]:
```
# task
text='Hello World World'
print(text.find('World'))
print(text.rfind('World'))
```

In [ ]:
```
"banana".index('xx')
```

In [ ]:
```
"banana".index('ya')
```

In [ ]:
```
"banana".count('a')
```

In [ ]:
```
"banana".count('Z')
```

In [ ]:
```
"python".startswith("W")
```

In [ ]:
```
"python".endswith("py")
```

In [ ]:
```
text = "Hello World"

print(text.replace("World", "Python"))
 # "Hello Python"
```

In [ ]:
```
" hello python ".strip()
```

In [ ]:
```
" python ".lstrip()
```

In [ ]:
```
" python ".rstrip()
```

In [ ]:
```
# Example 1:
text = " "
print(text.isalnum())
print(text.isalpha())
print(text.isdigit())
print(text.isupper())
print(text.islower())
print(text.isspace())
print(text.istitle())
```

In [ ]:
```
name = "Alice"
age = 25
print("Name: " + name + ", Age: " + str(age))
```

In [ ]:
```
print("Name:", name, "Age:", age)
```

In [ ]:
```
print("Name: {}, Age: {}".format(name,

                                 )
```

In [ ]:
```
print("Name: {0}, Age: {1}".format(name, age))
print("Name: {n}, Age: {a}".format(n=name, a=age))
```

In [ ]:
```
print(f"Name: {name}, Age: {age}")
```

In [ ]:
```
print("Name: %s, Age: %d" % (name, age))
```

4. List, Tuple, Set, Dictionary

In [ ]:
```
my_list = [10, 20, 30]
my_tuple = (10, 20, 30)
my_set = {10, 20, 30}
my_dict = {"name": "John", "age": 25}

print(my_list)
print(my_tuple)
print(my_set)
print(my_dict)
```

5. Creating Data Structures

In [ ]:
```
# List Creation:

List1= [ ]  # empty List
List2=list() # empty List
```

In [ ]:
```
print(List1)
```

In [ ]:
```
print(type(List1))
```

In [ ]:
```
# convert into list

a=4,5,5
```

In [ ]:
```
l=list(a)
```

In [ ]:
```
l
```

In [ ]:
```
# homogenous data type ( similar data types)
```

```
list2 = [10,30,60] # List of integers numbers
list3 = [10.77,30.66,60.89] # List of float numbers
list4 = ['one','two' , "three"] # List of strings
```

In [ ]:
```
# heterogenous data type ( Mixed Data types)

mixed_list = [101, "John", 75.5, True, None]
```

In [ ]:
```
mixed_list
```

In [ ]:
```
# mixed data types also used some other sequences in a list:

list7 = ['suresh', 25 ,(50, 100),[150, 90] , {'John' , 'David'}]
```

In [ ]:
```
list7
```

In [ ]:
```
nested_list = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

In [ ]:
```
nested_list
```

In [ ]:
```
#### Tuple Creation:

t1 = (1, 2, 3)
t2 = ()                 # empty tuple
t3 = (5,)               # single-element tuple (note the comma!)
t4 = tuple("abc")       # from iterable: ('a', 'b', 'c')
```

In [ ]:
```
t1
```

In [ ]:
```
t2
```

In [ ]:
```
t3
```

In [ ]:
```
t4
```

In [ ]:
```
# Homogenous Data Types:

tup2 = (10,30,60) # tuple of integers numbers
tup3 = (10.77,30.66,60.89) # tuple of float numbers
tup4 = ('one','two' , "three") # tuple of strings
```

In [ ]:
```
# Heterogenous Data Types:

tup6 = (100, 'Asif', 17.765) # Tuple of mixed data types
```

In [ ]:
```
tup7 = ('Asif', 25 ,[50, 100],[150, 90] , {'John' , 'David'} , (99,22,33)
```

In [ ]:
```
# Nested Tuples:

tup5 = ('john', 25 ,(50, 100),(150, 90)) # Nested tuples
```

In [ ]:
```
len(tup7) #Length of list
```

In [ ]:
```
#### creating a Set:

# Using curly braces
my_set = {1, 2, 3, 4}
```

In [ ]:
```
# Using the set() constructor
another_set = set([4, 5, 6])
```

In [ ]:
```
my_set
```

In [ ]:
```
another_set
```

In [ ]:
```
len(my_set) # length of the set
```

In [ ]:
```
my_set = {1,1,2,2,3,4,5,5}
my_set # Duplicate elements are not allowed
```

In [ ]:
```
myset1 = {1.79,2.08,3.99,4.56,5.45} # Set of float numbers
myset1
```

In [ ]:
```
myset2 = {'Asif' , 'John' , 'Tyrion'} # Set of Strings
myset2
```

In [ ]:
```
myset3 = {10,20, "Hola", (11, 22, 32)} # Mixed datatypes
myset3
```

In [ ]:
```
myset3 = {10,20, "Hola", [11, 22, 32]} # set doesn't allow mutable items
myset3
```

In [ ]:
```
my_set1 = set(('one' , 'two' , 'three' , 'four'))
my_set1
```

In [ ]:
```
mydict = dict() # empty dictionary
mydict
```

In [ ]:
```
mydict = {} # empty dictionary
mydict
```

In [ ]:
```
mydict = {1:'one' , 2:'two' , 3:'three'} # dictionary with integer keys
```

```
mydict
```

In [ ]:
```
mydict = dict({1:'one' , 2:'two' , 3:'three'}) # Create dictionary using
mydict
```

In [ ]:
```
mydict = {'A':'one' , 'B':'two' , 'C':'three'} # dictionary with characte
mydict
```

In [ ]:
```
mydict.keys() # Return Dictionary Keys using keys() method
```

In [ ]:
```
mydict.values() # Return Dictionary Values using values() method
```

In [ ]:
```
mydict.items() # Access each key-value pair within a dictionary
```

In [ ]:
```
mydict = {1:'one' , 2:'two' , 'A':['asif' , 'john' , 'Maria']} # dictiona
mydict
```

In [ ]:
```
mydict = {1:'one' , 2:'two' , 'A':['asif' , 'john' , 'Maria'], 'B':('Bat'
mydict
```

6. Data-Structure Indexing and Slicing

In [ ]:
```
values = [10, 20, 30, 40]
print(values[0])
print(values[1:3])

point = (100, 200, 300)
print(point[0])
print(point[:2])

student = {"name": "John", "age": 25}
print(student["name"])

numbers = {10, 20, 30}
# Set does not support indexing or slicing.
print(numbers)
```

In [ ]:
```
l=['Asif','Basit','John','Michele',1299,88.76]
```

In [ ]:
```
l[0]
```

In [ ]:
```
l[-1]
```

In [ ]:
```
l[0][1]  # nested indexing   -- Asif---> 's'
```

In [ ]:
```
mylist = ['one' , 'two' , 'three' , 'four' , 'five' , 'six' ,
          'seven' , 'eight']
```

In [ ]:
```
mylist[6]
```

In [ ]:
```
mylist[3:7] # slicing from location 3 to 7 excluding the last value
```

In [ ]:
```
mylist[:3] # Return first three items
```

In [ ]:
```
mylist[-3:] # Return last three items
```

In [ ]:
```
tup2[0] # Retreive first element of the tuple
```

In [ ]:
```
tup4[0] # Retreive first element of the tuple
```

In [ ]:
```
tup4[0][0] # Nested indexing
```

In [ ]:
```
tup4[-1] # Last item of the tuple
```

In [ ]:
```
# Tuple Slicing


mytuple = ('one' , 'two' , 'three' , 'four' , 'five' , 'six' , 'seven' ,
```

In [ ]:
```
mytuple
```

In [ ]:
```
mytuple[0:3]
```

In [ ]:
```
mytuple[0:3]
```

In [ ]:
```
mytuple[:3] # Return first three items
```

In [ ]:
```
mytuple[-3:] # Return last three items
```

In [ ]:
```
mydict = {1:'one' , 2:'two' , 3:'three' , 4:'four'}
mydict
```

In [ ]:
```
mydict[1] # Access item using key
```

In [ ]:
```
mydict.get(1) # Access item using get() method
```

In [ ]:
```
mydict1 = {'Name':'Asif' , 'ID': 74123 , 'DOB': 1991 , 'job' :'Analyst'}
mydict1
```

In [ ]:
```
mydict1['Name'] # Access item using key
```

In [ ]:
```
mydict1.get('job') # Access item using get() method
```

In [ ]:
```
v=[0,1,2,3,[8,{5:[67,89,90,{'area':'Southeast','city':'Bangalore'}]}]]
```

In [ ]:
```
v[4][1][5][3]['city'] # extract bangalore
```

List, Tuple, Set and Dictionary Operations

In [ ]:
```
# len(list)        Returns the number of elements in the list
mylist
```

In [ ]:
```
len(mylist)
```

In [ ]:
```
print("Before:",mylist) # existing list
```

In [ ]:
```
mylist.append('nine')
#  Add  an  item  to  the  end  of  the  list
print("After:",mylist)
```

In [ ]:
```
fruits = ["apple", "banana"]
fruits.append("cherry")
```

In [ ]:
```
fruits
```

In [ ]:
```
fruits.append(["orange", "grape"])
print(fruits)
```

In [ ]:
```
fruits = ["apple", "banana"]
fruits.insert(1, "grape")
print(fruits)
```

In [ ]:
```
fruits = ["apple", "banana", "cherry"]
```

In [ ]:
```
# Insert before the last element
fruits.insert(-1, "orange")
print(fruits)
```

In [ ]:
```
# Example 1: Extending with a List
fruits = ["apple", "banana"]
fruits.extend(["cherry", "orange"])
print(fruits)
```

In [ ]:
```
# Example 2: Extending with a String
```

```
chars = ["a", "b"]
chars.extend("cd")
print(chars)
```

In [ ]:
```
# Example 3 : Extending with Tuple:
numbers = [1, 2]
numbers.extend((3, 4))
print(numbers)  # [1, 2, 3, 4]
```

In [ ]:
```
a = [1, 2]
a.append([3, 4])
print(a)  # [1, 2, [3, 4]]

b = [1, 2]
b.extend([3, 4])
print(b)  # [1, 2, 3, 4]
```

In [ ]:
```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
print(fruits)
```

In [ ]:
```
numbers = [1, 2, 3, 2, 4]
numbers.remove(2)
print(numbers)
```

In [ ]:
```
animals = ["cat", "dog"]
animals.remove("lion")  # ❌ ValueError: list.remove(x): x not in list
```

In [ ]:
```
# example- 1 : pop last element:

fruits = ["apple", "banana", "cherry"]
removed_item = fruits.pop()
print(removed_item)  # 'cherry'
print(fruits)
```

In [ ]:
```
# example -2 : pop by index:

numbers = [10, 20, 30, 40]
removed_item = numbers.pop(1)
print(removed_item)  # 20
print(numbers)       # [10, 30, 40]
```

In [ ]:
```
# example - 3  IndexError

a = [1, 2, 3]
a.pop(5)  # ❌ IndexError: pop index out of range
```

In [ ]:
```
fruits = ["apple", "banana", "cherry"]
fruits.clear()
print(fruits)  # Output: []
```

In [ ]:
```
print(type(fruits))
```

In [ ]:
```
# example - 1 : basic usage index():
fruits = ["apple", "banana", "cherry", "banana"]
index_of_banana = fruits.index("banana")
print(index_of_banana)  # 1
```

In [ ]:
```
# example - 2 :  Specifying Start and End
```

In [ ]:
```
fruits = ["apple", "banana", "cherry", "banana"]
index_of_banana = fruits.index("banana", 2)
print(index_of_banana)  # 3

# The search starts from index 2, so it finds the second occurrence of "b
```

In [ ]:
```
fruits = ["apple", "banana", "cherry", "banana", "banana"]
banana_count = fruits.count("banana")
print(banana_count)  # 3
```

In [ ]:
```
l=[3,5,2,6,2,4,2,6]
l.count(2)
```

In [ ]:
```
# Example 1: Default Sort (Ascending)
numbers = [3, 1, 4, 2]
numbers.sort()
print(numbers)  # [1, 2, 3, 4]
```

In [ ]:
```
# Example 2: Sort in Descending Order
numbers = [3, 1, 4, 2]
numbers.sort(reverse=True)
print(numbers)  # [4, 3, 2, 1]
```

In [ ]:
```
fruits = ["apple", "banana", "cherry"]
fruits.reverse()
print(fruits)
```

In [ ]:
```
numbers = [3, 1, 4, 2]
numbers.reverse()
print(numbers)
```

In [ ]:
```
# Example 1: Default Sort (Ascending)
numbers = [3, 1, 4, 2]
sorted_numbers = sorted(numbers)
print(sorted_numbers)  # [1, 2, 3, 4]
```

In [ ]:
```
# Example 2: Sort in Descending Order

numbers = [3, 1, 4, 2]
sorted_numbers = sorted(numbers, reverse=True)
```

```
print(sorted_numbers)  # [4, 3, 2, 1]
```

In [ ]:
```
# Example 1: Joining Strings

words = ["Hello", "World"]
result = " ".join(words)
print(result)  # "Hello World"
```

In [ ]:
```
# Example 2 : Joining with a Comma

fruits = ["apple", "banana", "cherry"]
result = ", ".join(fruits)
print(result)  # "apple, banana, cherry"
```

In [ ]:
```
# join:
list1 = ['one', 'two', 'three', 'four']
list2 = ['five', 'six', 'seven', 'eight']

list3 = list1 + list2 # Join two lists by '+' operator
list3
```

In [ ]:
```
#  Example 1: Check Membership

fruits = ["apple", "banana", "cherry"]
result = "banana" in fruits
print(result)  # True
```

In [ ]:
```
# Example 2: Not in List:

result = "grape" in fruits
print(result)  # False
```

In [ ]:
```
mylist = ['one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight',
mylist1 = mylist # Create a new reference "mylist1"
id(mylist) , id(mylist1)
```

In [ ]:
```
mylist[2]=44
```

In [ ]:
```
mylist
```

In [ ]:
```
mylist1
```

In [ ]:
```
mylist2 = mylist.copy() # Create a copy of the list
```

In [ ]:
```
id(mylist2)
```

In [ ]:
```
mylist[0] = 1
```

In [ ]:
```
mylist
```

In [ ]:
```
mylist2
```

In [ ]:
```
L1 = [1,2,3,4]
```

In [ ]:
```
all(L1)
```

In [ ]:
```
L1 = [1,2,3,4,False]
```

In [ ]:
```
all(L1)
```

In [ ]:
```
L2=[]
```

In [ ]:
```
all(L2)
```

In [ ]:
```
L1 = [1,2,3,4,0]
```

In [ ]:
```
any(L1)
```

In [ ]:
```
l2=[]
```

In [ ]:
```
any(l2)
```

In [ ]:
```
# Packing
person = ("John", 25, "Developer")
```

In [ ]:
```
# Unpacking
name, age, role = person
print(name)  # John
```

In [ ]:
```
# Deleting Tuple:
del mytuple
```

In [ ]:
```
mytuple
```

In [ ]:
```
mytuple1 =('one', 'two', 'three', 'four', 'one', 'one', 'two', 'three')
```

In [ ]:
```
mytuple1.count('one') # Number of times item "one" occurred in the tuple.
```

In [ ]:
```
mytuple1.count('two') # Occurence of item 'two' in the tuple
```

In [ ]:
```
mytuple1.index('one') # Index of first element equal to 'one'
```

In [ ]:
```
mytuple1.index('two') # Index of first element equal to 'two'
```

In [ ]:
```
mytuple2 = (43,67,99,12,6,90,67)
```

In [ ]:
```
max(mytuple2)  # maximum value of the given tuple
```

In [ ]:
```
min(mytuple2)  # minimum value of the given tuple
```

In [ ]:
```
mytuple2 = (43,67,99,12,6,90,67)
sorted(mytuple2) # Returns a new sorted list and doesn't change original
```

In [ ]:
```
sorted(mytuple2, reverse=True) # Sort in descending order
```

In [ ]:
```
mytuple1
```

In [ ]:
```
'one' in mytuple1  # Check if 'one' exist in the list
```

In [ ]:
```
'ten' in mytuple1 # Check if 'ten' exist in the list
```

In [ ]:
```
'nine' not in mytuple1
```

In [ ]:
```
myset = {'one', 'two', 'three', 'four', 'five', 'six', 'seven', 'eight'}
```

In [ ]:
```
myset.add('NINE') # Add item to a set using add() method
myset
```

In [ ]:
```
myset.update(['TEN' , 'ELEVEN' , 'TWELVE']) # Add multiple item to a set
myset
```

In [ ]:
```
myset.remove('NINE') # remove item in a set using remove() method
myset
```

In [ ]:
```
myset.discard('TEN') # remove item from a set using discard() method
myset
```

In [ ]:
```
# Difference between remove () and Discard()

my_set = {1, 2, 3}

my_set.add(4)             # Adds an element
my_set.update([5, 6])     # Adds multiple elements
my_set.remove(2)          # Removes element; error if not found
my_set.discard(10)        # Removes element; no error if not found
```

In [ ]:
```
my_set
```

In [ ]:
```
my_set.clear()            # Removes all elements
```

In [ ]:
```
# Deleting a set:

del my_set
```

In [ ]:
```
my_set
```

In [ ]:
```
A = {1,2,3,4,5}
B = {4,5,6,7,8}
C = {8,9,10}
```

In [ ]:
```
# Union:

A | B # Union of A and B (All elements from both sets. NO DUPLICATES)
```

In [ ]:
```
A.union(B) # Union of A and B
```

In [ ]:
```
A.union(B) # Union of A and B
```

In [ ]:
```
A = {1,2,3,4,5}
B = {4,5,6,7,8}

# update()  # perform union operation and the result should be updated in

A.update(B,C)
A
```

In [ ]:
```
# Intersection()

A= {1,2,3,4,5}
B = {4,5,6,7,8}

A & B # Intersection of A and B (Common items in both sets)
```

In [ ]:
```
A.intersection(B) # Intersection of A and B
```

In [ ]:
```
A
```

In [ ]:
```
A.intersection_update(B)
A
```

In [ ]:
```
# Difference:

A= {1,2,3,4,5}
B = {4,5,6,7,8}
```

In [ ]:
```
A - B # set of elements that are only in A but not in B
```

In [ ]:
```
A.difference(B) # Difference of sets
```

In [ ]:
```
B- A # set of elements that are only in B but not in A
```

In [ ]:
```
B.difference(A)
```

In [ ]:
```
A= {1,2,3,4,5}
B = {4,5,6,7,8}
```

In [ ]:
```
A ^ B # Symmetric difference (Set of elements in A and B but not in both
```

In [ ]:
```
A.symmetric_difference(B) # Symmetric difference of sets
```

In [ ]:
```
A
```

In [ ]:
```
A.symmetric_difference_update(B)
A
```

In [ ]:
```
# Other Builtin functions

A
```

In [ ]:
```
sum(A)
```

In [ ]:
```
max(A)
```

In [ ]:
```
min(A)
```

In [ ]:
```
len(A)
```

In [ ]:
```
D= sorted(A,reverse=True)  # decending order
D
```

In [ ]:
```
sorted(D) # ascending order
```

In [ ]:
```
mydict1 = {'Name':'Asif' , 'ID': 12345 , 'DOB': 1991 , 'Address' : 'Hilsi
mydict1
```

In [ ]:
```
mydict1['DOB'] = 1992 # Changing Dictionary Items
mydict1['Address'] = 'Delhi'
mydict1
```

In [ ]:
```
dict1 = {'DOB':1995}
mydict1.update(dict1)
mydict1
```

In [ ]:
```
mydict1['Job'] = 'Analyst' # Adding items in the dictionary
mydict1
```

In [ ]:
```
mydict1.pop('Job') # Removing items in the dictionary using Pop method
mydict1
```

In [ ]:
```
mydict1.popitem() # A random item is removed
```

In [ ]:
```
mydict1
```

In [ ]:
```
del[mydict1['ID']] # Removing item using del method
mydict1
```

In [ ]:
```
mydict1.clear() # Delete all items of the dictionary using clear method
mydict1
```

In [ ]:
```
del mydict1 # Delete the dictionary object
mydict1
```

In [ ]:
```
mydict = {'Name':'Asif' , 'ID': 12345 , 'DOB': 1991 , 'Address' : 'Hilsin
mydict
```

In [ ]:
```
mydict1 = mydict # Create a new reference "mydict1"
```

In [ ]:
```
id(mydict) , id(mydict1) # The address of both mydict & mydict1 will be t
```

In [ ]:
```
mydict['Address'] = 'Mumbai'
```

In [ ]:
```
mydict1
```

In [ ]:
```
mydict
```

In [ ]:
```
# using copy() function:
mydict2 = mydict.copy() # Create a copy of the dictionary
```

In [ ]:
```
id(mydict2),id(mydict)
```

In [ ]:
```
mydict['Address'] = 'coimbatore'
```

In [ ]:
```
mydict2
```

In [ ]:
```
mydict
```

In [ ]:
```
mydict1 = {'Name':'Asif' , 'ID': 12345 , 'DOB': 1991 , 'Job': 'Analyst'}
mydict1
```

In [ ]:
```
'Name' in mydict1 # Test if a key is in a dictionary or not
```

In [ ]:
```
'salary' not in mydict1
```

In [ ]:
```
mydict1 = {'Name':'Asif' , 'ID': 12345 , 'DOB': 1991 , 'Address' : 'Hilsi
mydict1
```

In [ ]:
```
del mydict1['Name']
```

In [ ]:
```
mydict1
```

In [ ]:
```
del mydict1
```

7. Mutable vs Immutable

In [ ]:
```
# List is mutable
numbers = [10, 20, 30]
numbers[0] = 100
print(numbers)

# Dictionary is mutable
person = {"name": "John"}
person["name"] = "David"
print(person)

# Set is mutable
items = {1, 2, 3}
items.add(4)
print(items)

# Tuple is immutable
values = (10, 20, 30)
print(values)
# values[0] = 100   # TypeError
```

In [ ]:
```
# Original tuple immutability example (kept safe to run)
# mytuple[0] = 1 # Tuples are immutable which means we can't CHANGE tuple
#
```

8. if, if-else, if-elif-else, Nested Conditions

In [ ]:
```
age = 20

if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

In [ ]:
```
# 1. check if a number is positive:

num = 10
if num > 0:
    print("The number is positive")
```

In [ ]:
```python
# 2. Check if a number is even

num = 4
if num % 2 == 0:
    print("Even number")
```

In [ ]:
```python
# 3. Check if list is not empty
items = [1, 2, 3]
if items:
    print("List is not empty")
```

In [ ]:
```python
# 4. Check string length
name = "Python"
if len(name) > 5:
    print("Long name")
```

In [ ]:
```python
# 5. Check if character is a vowel
ch = 'a'
if ch in 'aeiou':
    print("Vowel")
```

In [ ]:
```python
# 1. even or odd number:

num = 4
if num % 2 == 0:   # 4 % 2==0   # 0==0
    print("Even")
else:
    print("Odd")
```

In [ ]:
```python
# 2. Check Positive or Negative
x = -5
if x >= 0:
    print("Positive")
else:
    print("Negative")
```

In [ ]:
```python
# 3. String Length Check
name = "AI"
if len(name) > 3:   # 2 >3
    print("Long name")
else:
    print("Short name")
```

In [ ]:
```python
# 4. Check Voting Eligibility
age = 16
if age >= 18:
    print("Eligible to vote")
else:
    print("Not eligible to vote")
```

In [ ]:
```python
# 5. Check Password Match:
```

```
password = "admin123"
confirm = "admin123"
if password == confirm:
    print("Password matched")
else:
    print("Password mismatch")
```

In [ ]:
```
# 1. example: Grading System:
marks = 76

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Grade D")
```

In [ ]:
```
# example : 2 BMI Category:
bmi = 22.5

if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal weight")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

In [ ]:
```
# elif can be used to check alternate conditions
country=input("Name of the country:")
if country==country.upper():
    print("The font entered is ALL CAPITAL")
elif country==country.lower():
    print("The font is all lowercase.")
elif country==country.capitalize():
    print("The font received is appropriate.")
else:
    print("Mixed case received")



print("One of the 4 blocks of code was executed.")
```

In [ ]:
```
age = 25
income = 40000

if age > 18 and income > 30000:
    print("Eligible for loan")
```

In [ ]:
```python
city = "Chennai"

if city == "Chennai" or city == "Bangalore":
    print("City is in South India")
```

In [ ]:
```python
is_logged_in = False

if not is_logged_in:
    print("Please log in")
```

In [ ]:
```python
# Example-1: Age and Income Eligibility for Loan

age = eval(input('enter your age'))
income = eval(input('enter your salary'))
credit_score = eval(input('enter your score'))

if age >= 18 and income > 30000 and credit_score >= 650:
    print("Eligible for loan")
elif age >= 18 and income > 30000:
    print("Eligible for loan with high interest")
else:
    print("Not eligible for loan")
```

In [ ]:
```python
# example-2 : check if a number is within a range:
number = 25

if (number >= 10 and number <= 20) or (number >= 30 and number <= 40):
    print("Number is in the range 10-20 or 30-40")
else:
    print("Number is outside the ranges")
```

In [ ]:
```python
# example -3 : check the given year is leap or not:
year = int(input('enter the year:'))
if (year%4==0 and year%100!=0) or (year%400==0):
  print(f'{year} is leap year')
else:
  print(f'{year} is not a leap year')
```

In [ ]:
```python
# combining and,or,not using if statement:
age = 30
has_license = True
citizen = False

if (age >= 18 and has_license) or not citizen:
    print("Eligible to drive")
else:
    print("Not eligible to drive")
```

In [ ]:
```python
# 1.  Nested if for Checking Multiple Conditions (Age and Height)
age = 20
height = 170
```

```
if age >= 18:
    if height >= 160:
        print("You are an adult and tall enough for the ride.")
    else:
        print("You are an adult but not tall enough for the ride.")
else:
    print("You are not an adult.")
```

In [ ]:
```
#nested if structure
balance = float(input('Enter your bank balance:'))
withdraw = float(input('Enter the amount you want to withdraw: '))
if withdraw <= balance:
  if withdraw > 500:
    print('This transaction is approved')
  else:
    print('transaction requires manager approval')
else:
  print('Insufficient funds.')
```

In [ ]:
```
if num>0:
    if num%2==0:
        print('positive even')
    else:
        print('positive odd')
else:
    if num%2==0:
        print('negative even')
    else:
        print('negative odd')
```

In [ ]:
```
temp = float(input('enter the temperature:'))
if temp>0:
  if temp >=30:#temp>0 and temp>30
    print("It is extremely hot")
  else:#temp>0 and temp<30
    print('Hot')
elif temp==0:
    print('neutral')
else:
  if temp<=-30:#temp<0 and temp<=-30
    print('It is extremely cold')
  else:#temp<0 and temp>=-30
    print('Cold')
```

In [ ]:
```
fruits = ['apple', 'banana', 'cherry']

if 'banana' in fruits:
    print("Banana is in the list.")
else:
    print("Banana is not in the list.")
```

In [ ]:
```python
colors = ('red', 'green', 'blue')

if 'green' in colors:
    print("Green is in the tuple.")
else:
    print("Green is not in the tuple.")
```

In [ ]:
```python
### 3. Using if with a Dictionary

person = {
    'name': 'John',
    'age': 30,
    'city': 'New York'
}

if 'age' in person:
    print("The key 'age' is present in the dictionary.")
else:
    print("The key 'age' is not present in the dictionary.")
```

9. while and for Loops

In [ ]:
```python
i = 1
while i <= 3:
    print(i)
    i += 1

for i in range(1, 4):
    print(i)
```

In [ ]:
```python
# Example -1 :
number = 1
while number <= 5:
    print("Number is:", number)
    number += 1
```

In [ ]:
```python
# Example - 2: Using the else Statement with While Loop in Python


count = 1
while count <= 5:
    print(count)
    count += 1
else:
    print("Loop completed without interruption.")
```

In [ ]:
```python
age = 32

# The test condition is always True
while age > 18:
```

```
    print('You can vote')
```

In [ ]:
```
# Simple while loop
i=1 # counter initialized
while i<6:
    print('While condition is satisfied for',i) # block of code indented
    i=i+1 # incrementer
print("Loop is exited") # outside the while block-of-code
```

In [ ]:
```
print("Even numbers between 0 and 10:")
c=0
while c<11:
    print(c)
    c+=2 # operator += equivalent to incrementing by 2
```

In [ ]:
```
# While loop with else statement
i=3
while i<6:
    i+=1
    print(i)

else:
    print('While loop condition is false')
```

In [ ]:
```
count = 1
while (count < 3):
    count = count + 1
    print("Data science")
```

In [ ]:
```
# Identify whether a given number is prime or composite
num=int(input("Enter a number"))
i=1 # iterator
count=0 # no. of factors
while i<=num/2:
    if num%i==0:
      count += 1
    i += 1

if count==1:
  print("Prime")
else:
  print("Composite")
```

In [ ]:
```
# Example list of numbers
numbers = [1, 2, 3, 4, 5]
# Using a for loop to iterate through the list
for number in numbers:
    print(number)
```

In [ ]:
```
fruits=['Apple','Banana','Mango','Watermelon']
for i in fruits:
    print(i)
```

In [ ]:
```
s='#12th Main Road'
for i in s:
    print(i) # string is a variable made up of characters in sequence
```

In [ ]:
```
# Separating the above string by spaces
for i in s.split(): # ['#12th','Main','Road']
    print(i)
```

In [ ]:
```
## Range function
for i in range(10):
# range is a generator function which
#yields numbers from specified start to end point
    print(i)
```

In [ ]:
```
for i in range(1,20,2):
    # lower limit,upper limit,step
    print(i)
```

In [ ]:
```
for i in range(3):           # Outer loop
    for j in range(2):       # Inner loop
        print(f"i = {i}, j = {j}")
```

In [ ]:
```
# Nested loops
a=['red','blue','green']
b=['Apple','Banana','Orange']
for i in a: ## outer loop
    for h in b: ## inner loop
        print(i,h)
```

In [ ]:
```
# Another example of nesting
course=['Maths','Data Science','Economics','Social Sciences']
size=[78,90,56,90] # size of batches A,B,C,D
batch=('A','B','C','D')
```

In [ ]:
```
for i in course: # Outer loop
    for k in size: # inner loop
        b_loc=size.index(k)
        b = batch[b_loc]
        print(f"Batch {b} of size {k} shall attend {i}")
```

10. break, continue, pass

In [ ]:
```
for i in range(1, 6):
    if i == 3:
        break
    print(i)

for i in range(1, 6):
```

```
    if i == 3:
        continue
    print(i)

for i in range(3):
    pass
```

In [ ]:
```
# Example - 1
number = 1
while number <= 5:
    if number == 3:
        print("Stopping loop at number 3")
        break
    print("Number is:", number)
    number += 1
```

In [ ]:
```
# Example - 2

nums = [25,40,25,80,75,25,88,25,24,25,77,25,56,25,10,25]
print(nums)
print(nums.count(25))
```

In [ ]:
```
# while loop with break statement
# break command stops the loop prematurely

while 25 in nums: # membership check
    nums.remove(25)
    c = nums.count(25)
    if c==5:
        break # exits the while loop without further checks

print(nums)
print("Count of 25s in the list is now",c)
```

In [ ]:
```
# Example - 1:
i = 0
while i < 5:
    i += 1
    if i == 3:
        continue  # skip printing 3
    print(i)
```

In [ ]:
```
# Example - 2:
# While loop with continue statement.
i=0
while i<8:
    i+=1 # incrementer provided early
    sq = i**2
    if 10<sq<30:
        print("*********Iteration skipped here*******")
        continue
    print(f"Square of {i} = {i**2}")
```

In [ ]:
```
# using Break:
languages = ['Swift', 'Python', 'Go', 'C++']

for lang in languages:
    if lang == 'Go':
        break
    print(lang)
```

In [ ]:
```
# using continue:
languages = ['Swift', 'Python', 'Go', 'C++']

for lang in languages:
    if lang == 'Go':
        continue
    print(lang)
```

11. Functions and Arguments

In [ ]:
```
def add(a, b):
    return a + b

print(add(2, 3))
```

In [ ]:
```
# Creating a Function in Python:

def fun():
    print("Good Morning")
```

In [ ]:
```
# Calling Functions in Python

fun()
```

In [ ]:
```
def greet():
    print("Hello, welcome to Python!")

greet()  # Call the function
```

In [ ]:
```
def get_pi():
    return 3.14159

pi_value = get_pi()
print("Value of Pi is:", pi_value)
```

In [ ]:
```
def greet_user(name):
    print(f"Hello, {name}! Have a great day.")

greet_user("john")
```

In [ ]:
```python
#### 4) Function with Argument and Return Value:
def multiply(x, y):
    return x * y

result = multiply(4, 5)
print("Multiplication result is:", result)
```

In [ ]:
```python
#1. Default Arguments
def greet(name="Guest"):
    """
    This function greets the person whose name is passed as an argument.
    If no name is provided, it uses 'Guest' as the default.
    """
    print(f"Hello, {name}! Welcome to the Python world.")

# Calling the function without an argument
greet()
# Calling the function with an argument
greet("Raman")
```

In [ ]:
```python
# 2. Keyword Arguments


def introduce(name, age, city):
    """
    This function introduces a person with their name, age, and city.
    """
    print(f"Hello, my name is {name}. I am {age} years old and I live in
# Calling the function with keyword arguments
introduce(name="Raman", age=30, city="New York")
```

In [ ]:
```python
# 3. Positional Arguments
def add_numbers(a, b):
    """
    This function takes two numbers and returns their sum.
    """
    return a + b
# Calling the function with positional arguments
result = add_numbers(5, 10)
print("The sum is:", result)
```

In [ ]:
```python
# 4. Arbitrary Keyword  Arguments

# 1. *args → Non-keyworded Variable-Length Arguments

def total(*args):
    print("Arguments received:", args)
    return sum(args)

print("Total =", total(10, 20, 30, 40))
```

In [ ]:
```python
# 2. **kwargs → Keyworded Variable-Length Arguments
```

```
def display_info(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} = {value}")

display_info(name="john", course="Python", duration="30 Days")
```

In [ ]:
```
#  Combine *args and **kwargs

def mix_example(*args, **kwargs):
    print("Positional args:", args)
    print("Keyword args:", kwargs)

mix_example(1, 2, 3, name="Luqman", course="Data Science")
```

12. Lambda, filter(), map(), reduce()

In [ ]:
```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

print(list(filter(lambda x: x % 2 == 0, numbers)))
print(list(map(lambda x: x * 2, numbers)))
print(reduce(lambda a, b: a + b, numbers))
```

In [ ]:
```
addition = lambda a : a + 10 # This lambda function adds value 10 to an a
print(addition(5))
```

In [ ]:
```
product = lambda a, b : a * b
print(product(5, 6))
```

In [ ]:
```
addition = lambda a, b, c : a + b + c #This lambda function takes three a
print(addition(5, 6, 2))
```

In [ ]:
```
res = (lambda *args: sum(args)) # This lambda function can take any numbe
res(10,20) , res(10,20,30,40) , res(10,20,30,40,50,60,70)
```

In [ ]:
```
res1 = (lambda **kwargs: sum(kwargs.values())) # This lambda function can
res1(a = 10 , b= 20 , c = 30) , res1(a = 10 , b= 20 , c = 30, d = 40 , e
```

In [ ]:
```
# normal def method using filter():

list1 = [1,2,3,4,5,6,7,8,9]
def odd(n):
    if n%2 ==1: return True
    else: return False

odd_num = list(filter(odd,list1))
odd_num
```

In [ ]:
```
# lambda using filter ():

list1 = [1,2,3,4,5,6,7,8,9]
# The below Filter function filters "list1" and passes all odd numbers us
odd_num = list(filter(lambda n: n%2 ==1 ,list1))
odd_num
```

In [ ]:
```
# def method using map () :

def twice(n):
    return n*2

doubles = list(map(twice,odd_num))
doubles
```

In [ ]:
```
# lambda method using map ():

doubles = list(map(lambda n:n*2,odd_num))
doubles
```

In [ ]:
```
# def method using reduce ():

from functools import reduce

def add(a,b):
    return a+b

sum_all = reduce(add,doubles)
sum_all
```

In [ ]:
```
#The below reduce() function will perform sum of all items in the list us
sum_all = reduce(lambda a,b : a+b,doubles)
sum_all
```